



CSE Department – Faculty of Engineering - MSA  
Spring 2026

# Contact Manager

Software Testing, Validation and Verification

(CSE265)

Under the supervision of:

Dr. Gehad Mohey

**Student Name:**

**ID:**

Mazen Yasser

247861

Jasmine Ali

240247

Sandy Atef

245077

Mariam Mohamed Reyad

245171

Marwan Ehab

248889

## Contents

|                                       |    |
|---------------------------------------|----|
| 1. Introduction .....                 | 3  |
| 2. Test Plan .....                    | 4  |
| 2.1 Objectives.....                   | 4  |
| 2.2 Scope .....                       | 4  |
| 2.3 Features to be Tested .....       | 4  |
| 2.4 Risks Analysis .....              | 5  |
| 3. Static Design Review .....         | 5  |
| 3.1 Objective .....                   | 6  |
| 3.2 Review Findings.....              | 6  |
| 4. Static Implementation Review ..... | 9  |
| 4.1 Objective .....                   | 9  |
| 4.2 Review Findings.....              | 9  |
| 5. Test Design .....                  | 11 |
| 5.1 Unit Testing .....                | 11 |
| 5.2 Integration Testing.....          | 15 |
| 5.3 System Testing .....              | 15 |
| 6. Test Implementation and Logs ..... | 17 |
| 6.1 Test Execution Results .....      | 17 |
| 6.2 Defect Log.....                   | 19 |
| 7. Conclusion.....                    | 21 |
| 8. References.....                    | 9  |

# 1. Introduction

The purpose of this report is to present the software testing activities performed for the Contact Manager System. The project is a C++ console-based application designed to allow users to manage contact information through several core functionalities including **adding, searching, updating, deleting, and viewing contacts**.

This report documents the testing process carried out to verify that the system satisfies both functional and non-functional requirements. Different testing techniques and levels were applied, including **static testing, unit testing, integration testing, and system testing**, in order to ensure the reliability, correctness, and maintainability of the application.

The report also includes reviews of the software design and implementation, test case design, execution results, and defect observations. The overall objective is to evaluate the quality of the system and identify any issues before final deployment.

## 2. Test Plan

### 2.1 Objectives

- To verify all functional requirements specified in the SRS, including adding, viewing, searching, updating, and deleting contacts.
- To ensure data consistency on JSON file.
- To verify all non-functional requirements stated in the SRS.
- To identify bugs and inconsistencies

### 2.2 Scope

The testing scope for this project includes:

- System testing, unit testing, integration testing, static testing, and dynamic testing.
- Static review and dynamic execution of all project-owned modules are included in scope, but external libraries are excluded from testing.
- Performance testing is excluded.

Testing types included:

- Static Testing
- Dynamic Testing
- Unit Testing
- Integration Testing
- System Testing

### 2.3 Features to be Tested

- System Initialization
- Main Menu Navigation
- Add Contact
- Search Contact
- Update Contact
- Delete Contact
- View All Contacts
- Input Validation
- Data Persistence

## 2.4 Risks Analysis

- Missing JSON file or data corruption in the JSON file due to unexpected termination during add, update, or delete operations .  
Likelihood: Medium  
Impact: Critical
- Failure to validate empty or invalid fields (e.g., missing name or invalid numbers).  
Likelihood: High  
Impact: Critical
- Duplicate contacts due to lack of uniqueness constraints.  
Likelihood: High  
Impact: High
- Duplicate assigned unique IDs.  
Likelihood: Low  
Impact: Critical
- Failure in search leading to missing or wrong results.  
Likelihood: Medium  
Impact: High
- Data inconsistency between shown and stored data.  
Likelihood: Medium  
Impact: Critical

## 3. Static Design Review

### 3.1 Objective

The purpose of the static design review is to evaluate the software design documents and UML diagrams without executing the program.

### 3.2 Review Findings

- UML Class Diagram Review

| Requirement Reference/ Source | UML Class Diagram Observation                                                                                   | Review Finding / Status | Severity | Priority |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------|----------|----------|
| FR-IN-01/02                   | System does load all contacts at startup, and if the file does not exist, the system creates an empty one       | PASS                    | CRITICAL | HIGH     |
| FR-MM-01                      | ContactConsole is well designed and implemented as it has every function and shows them all                     | PASS                    | MODERATE | HIGH     |
| FR-AC-01/02/03                | ContactService.addContact takes (name , phone number), assigns a unique ID, and stores the contact at JSON file | PASS                    | CRITICAL | HIGH     |
| FR-VC-01/02                   | viewContacts does display all contacts, with each one having (ID, Name, Phone Number)                           | PASS                    | HIGH     | HIGH     |
| FR-UC-01/02/03                | updateContact does select contact by ID, modifies the information, and stores the new changes                   | PASS                    | CRITICAL | MODERATE |
| FR-DC-01/02/03                | deleteContact does select contact by ID, removes it, and update the JSON file                                   | PASS                    | MODERATE | HIGH     |

Table 3.21

- UML IN Sequence Diagram Review

| Requirement Reference / Source | UML Sequence Diagram Observation                                    | Review Finding / Status | Severity | Priority |
|--------------------------------|---------------------------------------------------------------------|-------------------------|----------|----------|
| FR-IN-01                       | The system loads all contacts from the JSON file at startup         | PASS                    | CRITICAL | HIGH     |
| FR-IN-02                       | The system creates a new JSON file if there was not an existing one | PASS                    | CRITICAL | HIGH     |

Table 3.22

- UML AC Sequence Diagram Review

| Requirement Reference / Source | UML Sequence Diagram Observation                                                                     | Review Finding / Status | Severity | Priority |
|--------------------------------|------------------------------------------------------------------------------------------------------|-------------------------|----------|----------|
| FR-AC-01                       | The user provides both the name and phone number, and the system does check if they are valid inputs | PASS                    | HIGH     | HIGH     |
| FR-AC-02                       | The system does assign unique IDs to each contact                                                    | PASS                    | CRITICAL | HIGH     |
| FR-AC-03                       | The system does store the contact to the JSON file                                                   | PASS                    | CRITICAL | HIGH     |

Table 3.23

- UML VC Sequence Diagram Review

| Requirement Reference / Source | UML Sequence Diagram Observation                      | Review Finding / Status | Severity | Priority |
|--------------------------------|-------------------------------------------------------|-------------------------|----------|----------|
| FR-VC-01                       | The system does show all stored contacts              | PASS                    | HIGH     | HIGH     |
| FR-VC-02                       | Each contact does contain: ID, name, and phone number | PASS                    | MODERATE | HIGH     |

Table 3.24

- UML SC Sequence Diagram Review

| Requirement Reference / Source | UML Sequence Diagram Observation                             | Review Finding / Status | Severity | Priority |
|--------------------------------|--------------------------------------------------------------|-------------------------|----------|----------|
| FR-SC-01                       | The system does require a name or a part of it from the user | PASS                    | MODERATE | MODERATE |
| FR-SC-02                       | The system does return all matching contacts                 | PASS                    | MODERATE | HIGH     |

Table 3.25

- UML UC Sequence Diagram Review

| Requirement Reference / Source | UML Sequence Diagram Observation                                 | Review Finding / Status | Severity | Priority |
|--------------------------------|------------------------------------------------------------------|-------------------------|----------|----------|
| FR-UC-01                       | System does select contacts by ID                                | PASS                    | MODERATE | MODERATE |
| FR-UC-02                       | User enters new information and the system checks if it is valid | PASS                    | HIGH     | HIGH     |
| FR-UC-03                       | The system does modify and save changes                          | PASS                    | CRITICAL | HIGH     |

Table 3.26

- UML DC Sequence Diagram Review

| Requirement Reference / Source | UML Sequence Diagram Observation     | Review Finding / Status | Severity | Priority |
|--------------------------------|--------------------------------------|-------------------------|----------|----------|
| FR-DC-01                       | The system selects a contact by ID   | PASS                    | MODERATE | MODERATE |
| FR-DC-02                       | System removes contact from the list | PASS                    | HIGH     | HIGH     |
| FR-DC-03                       | System updates JSON file             | PASS                    | HIGH     | HIGH     |

Table 3.27



## 4. Static Implementation Review

### 4.1 Objective

The purpose of the implementation review is to inspect the source code for defects, inconsistencies, and coding standard violations without running the application.

### 4.2 Review Findings

- ContactConsole

| Test Basis Reference                             | Module Observation                                                                                                                                                                           | Review Finding / Status | Severity        | Priority        |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------|-----------------|
| <b>Class Diagram - ContactConsole visibility</b> | void showMenu(), void handleAdd(), void handleView(), void handleSearch(), void handleUpdate(), void handleDelete() are all declared as private while the class diagram marks them as public | <b>Fail</b>             | <b>Moderate</b> | <b>Moderate</b> |
| <b>Sequence Diagram - Initialization flow</b>    | ContactConsole calls initialize() on ContactService, then calls load() on ContactRepository                                                                                                  | <b>Pass</b>             |                 |                 |
| <b>Sequence Diagram - Add contact flow</b>       | add Contact() validates through Utils before continuing                                                                                                                                      | <b>Pass</b>             |                 |                 |
| <b>Sequence Diagram - View contacts flow</b>     | viewContact() calls getAll() on repository                                                                                                                                                   | <b>Pass</b>             |                 |                 |
| <b>Sequence Diagram - Search flow</b>            | search() calls getAll() then loops through each contact using Utils::contains()                                                                                                              | <b>Pass</b>             |                 |                 |
| <b>Sequence Diagram - Delete flow</b>            | deleteContact() calls remove(id) on repository                                                                                                                                               | <b>Pass</b>             |                 |                 |

Table 4.21

- **ContactService**

| Test Basis Reference                              | Module Observation                                                                                                                             | Review Finding / Status | Severity        | Priority        |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------|-----------------|
| <b>Class diagram-void initialize()</b>            | void initialize() is shown in the code file but not present in the class diagram                                                               | <b>Fail</b>             | <b>Moderate</b> | <b>Moderate</b> |
| <b>Class diagram - visibility</b>                 | all members and methods have matching visibility between diagram and code                                                                      | <b>Pass</b>             |                 |                 |
| <b>Class diagram - ContractService operations</b> | addContact(), updateContact(), search(), viewContracts(), generateId(), deleteContact() are implemented and return the intended domain objects | <b>Pass</b>             |                 |                 |
| <b>Sequence diagram - file handling</b>           | Diagram shows alt [File not found] then initialize empty list                                                                                  | <b>Pass</b>             | <b>Low</b>      | <b>Low</b>      |
| <b>Sequence diagram - error handling</b>          | if load() fails or is denied, no error is shown reporting back to ContactConsole                                                               | <b>Fail</b>             | <b>High</b>     | <b>High</b>     |
| <b>Sequence diagram - Invalid entry</b>           | diagram shows alt [Invalid input] then failure is returned to ContactConsole                                                                   | <b>Pass</b>             |                 |                 |
| <b>Sequence diagram - duplicates check</b>        | no check if contact with same name or phone already exist before belong added                                                                  | <b>Fail</b>             | <b>Moderate</b> | <b>Moderate</b> |
| <b>Sequence diagram - ID check</b>                | diagram shoes alt [ID exists] then save() then success and if [Invalid ID] then failure                                                        | <b>Pass</b>             | <b>Low</b>      | <b>Low</b>      |

Table 4.22

- **ContactRepository**

| Test Basis Reference                                | Module Observation                                                                                         | Review Finding / Status | Severity    | Priority    |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------|-------------|-------------|
| <b>Class diagram - visibility</b>                   | all members and methods have matching visibility between diagram and code                                  | <b>Pass</b>             |             |             |
| <b>Class diagram - ContactRepository operations</b> | getAll(), add(), update(), remove(), load(), save() are implemented and return the intended domain objects | <b>Pass</b>             |             |             |
| <b>Sequence diagram - transaction</b>               | if save() fails after add(), there is no rollback option                                                   | <b>Fail</b>             | <b>High</b> | <b>High</b> |
| <b>Sequence diagram - Save Operation</b>            | save() is only called after update()                                                                       | <b>Pass</b>             |             |             |

Table 4.23

- **Contact**

| Test Basis Reference                       | Module Observation                                                        | Review Finding / Status | Severity   | Priority   |
|--------------------------------------------|---------------------------------------------------------------------------|-------------------------|------------|------------|
| <b>Class diagram - visibility</b>          | all members and methods have matching visibility between diagram and code | <b>Pass</b>             |            |            |
| <b>Class diagram - Constructor use</b>     | const std::string& for parameters                                         | <b>Pass</b>             |            |            |
| <b>Class diagram - setName(), setPhone</b> | accepts any string values to be entered without validation                | <b>Fail</b>             | <b>Low</b> | <b>Low</b> |

Table 4.24

- **Utils**

| Test Basis Reference                        | Module Observation                                                           | Review Finding / Status | Severity    | Priority    |
|---------------------------------------------|------------------------------------------------------------------------------|-------------------------|-------------|-------------|
| <b>Class diagram - visibility</b>           | isValidName(), isValidPhone(), contains() match between the diagram and code | <b>Pass</b>             |             |             |
| <b>Class diagram - toLower() visibility</b> | toLowerCase() is shown only in the code and not the diagram                  | <b>Fail</b>             |             |             |
| <b>Class diagram - toLower()</b>            | toLowerCase() takes std::string but tries to modify text                     | <b>Fail</b>             | <b>High</b> | <b>High</b> |
| <b>Class diagram - isValidPhone()</b>       | no length validation, therefore any number of characters can be entered      | <b>Fail</b>             | <b>Low</b>  | <b>Low</b>  |

Table 4.25

## 5. Test Design

### 5.1 Unit Testing

#### For ADD Contact:

Taking requirement FR-AC, we can deduce:

1. Verify that the system adds a contact when the name and phone number are valid.
2. Verify that the system does not add a contact when the name is empty.
3. Verify that the system does not add a contact when the phone number is empty.
4. Verify that the system does not add a duplicate contact.

#### Deduced Decision Table:

| Conditions     | 1 | 2 | 3 | 4 |
|----------------|---|---|---|---|
| Name Valid     | Y | N | Y | Y |
| Phone Valid    | Y | Y | N | Y |
| Contact exists | N | N | N | Y |
| <b>Action</b>  |   |   |   |   |
| Add Contact    | Y |   |   |   |
| Show Error     |   | Y | Y | Y |

Fig 5.11

**Deduced Test Cases:**

| <b>Test Case ID</b> | <b>Requirement Reference</b> | <b>Description</b>                                                        | <b>Test Data</b>                                                    | <b>Expected Outcome</b>                                |
|---------------------|------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------|
| UT-01               | FR-AC-01                     | Verify that the system adds a contact with valid data.                    | Name: Ali, Phone: 01012345678                                       | Contact is added successfully.                         |
| UT-02               | FR-AC-01                     | Verify that the system does not add a contact with an empty name.         | Name: empty, Phone: 01012345678                                     | Add operation fails and an error message is displayed. |
| UT-03               | FR-AC-01                     | Verify that the system does not add a contact with an empty phone number. | Name: Ali, Phone: ""                                                | Add operation fails and an error message is displayed. |
| UT-04               | FR-AC-02                     | Verify that the system assigns unique IDs to each contact                 | 3 different contacts with different names and phones                | Each contact is assigned a unique ID                   |
| UT-05               | FR-AC-01                     | Verify that the phone format is valid                                     | Name: Ali, Phone: abd123                                            | Contact is not added and invalid error is displayed    |
| UT-06               | FR-AC-01                     | Verify that the system does not add a duplicate contact.                  | Existing contact: Ali, 01012345678<br>New contact: Ali, 01012345678 | Contact is not added and duplicate error is displayed. |

Fig 5.12

**For Search Contact:**

Taking requirement FR-SC, we can deduce:

1. Verify that the system finds an existing contact.
2. Verify that the system does not find a non-existing contact.
3. Verify that the system handles empty search input.

**Deduced Decision Table:**

| Conditions                | 1 | 2 | 3 |
|---------------------------|---|---|---|
| Search input valid        | Y | Y | N |
| Contact exists            | Y | N | - |
| Action                    |   |   |   |
| Display contact           | ✓ |   |   |
| Display not found message |   | ✓ |   |
| Display input error       |   |   | ✓ |

Table 5.13

**Deduced Test Cases:**

| Test Case ID | Requirement Reference | Description                                                        | Test Data         | Expected Outcome                        |
|--------------|-----------------------|--------------------------------------------------------------------|-------------------|-----------------------------------------|
| UT-07        | FR-SC-01              | Verify that the system finds an existing contact by name.          | Search name: Ali  | Ali's contact details are displayed.    |
| UT-09        | FR-SC-01              | Verify that the system shows not found for a non-existing contact. | Search name: Omar | System displays "Contact not found."    |
| UT-08        | FR-SC-01              | Verify that the system searches with partial part of name          | Search name: Al   | System displays contacts containing Al. |

Table 5.14

**For Update Contact:**

Taking requirement FR-UC, we can deduce:

1. Verify that the system updates an existing contact with valid new data.
2. Verify that the system does not update a non-existing contact.
3. Verify that the system does not update a contact with an empty name.
4. Verify that the system does not update a contact with an empty phone number.

**Deduced Test Cases:**

| Test Case ID | Requirement Reference | Description                                                         | Test Data                                        | Expected Outcome                                   |
|--------------|-----------------------|---------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|
| UT-12        | FR-UC-01              | Verify that the system updates an existing contact with valid data. | Existing: Ali, 01012345678 New phone: 0111111111 | Contact is updated successfully.                   |
| UT-13        | FR-UC-01              | Verify that the system does not update a non-existing contact.      | Search name: Mona                                | Update fails and "Contact not found" is displayed. |

Table 5.15

**For Delete Contact:**

Taking requirement FR-DC, we can deduce:

1. Verify that the system deletes an existing contact.
2. Verify that the system does not delete a non-existing contact.
3. Verify that the system confirms the deletion.

**Deduced Test Cases:**

| Test Case ID | Requirement Reference | Description                                                    | Test Data               | Expected Outcome                                   |
|--------------|-----------------------|----------------------------------------------------------------|-------------------------|----------------------------------------------------|
| UT-10        | FR-DC-01              | Verify that the system deletes an existing contact.            | Existing contact ID     | Contact is removed from the contact list.          |
| UT-11        | FR-DC-01              | Verify that the system does not delete a non-existing contact. | Non-Existing contact ID | Delete fails and "Contact not found" is displayed. |

Table 5.16

## 5.2 Integration Testing

| Test Case ID | Test Case                 | Steps                            | Expected Result              |
|--------------|---------------------------|----------------------------------|------------------------------|
| IT-01        | Add contact integration   | Console → Service → Save JSON    | Contact stored in file       |
| IT-02        | View contacts integration | Load from JSON → display         | All contacts shown correctly |
| IT-03        | Search integration        | Input name → service → file data | Correct match returned       |
| IT-04        | Update integration        | Modify contact → save → reload   | Updated data appears in file |
| IT-05        | Delete integration        | Remove contact → update file     | Contact removed from JSON    |

Table 5.21

## 5.3 System Testing

| Test Case ID | Test Case              | Steps                                 | Expected Result                                 |
|--------------|------------------------|---------------------------------------|-------------------------------------------------|
| ST-01        | Full workflow test     | Add → View → Search → Update → Delete | All operations work correctly                   |
| ST-02        | Startup test           | Run program                           | Contacts loaded from JSON or empty list created |
| ST-03        | Data persistence test  | Add contact → restart program         | Contact still exists                            |
| ST-04        | End-to-end update test | Add → update → view                   | Updated data shown correctly                    |
| ST-05        | End-to-end delete test | Add → delete → view                   | Contact no longer exists                        |
| ST-06        | Menu navigation test   | Use all menu options                  | No crashes, correct routing                     |

Table 5.31

## 5.4 Dynamic Testing

| Test Case ID | Test Case                            | Input                              | Expected Result            |
|--------------|--------------------------------------|------------------------------------|----------------------------|
| DT-01        | Add contact via menu                 | 1 → name + phone                   | Contact added successfully |
| DT-02        | View contacts                        | Menu option 2                      | All contacts displayed     |
| DT-03        | Search contact                       | Menu option 3 + name               | Matching results shown     |
| DT-04        | Update contact                       | Menu option 4 + ID                 | Data updated successfully  |
| DT-05        | Delete contact                       | Menu option 5 + ID                 | Contact removed            |
| DT-06        | Invalid menu choice                  | Input: 99                          | Error message shown        |
| DT-07        | Empty input handling                 | Empty name/phone                   | Validation error shown     |
| DT-08        | Manual JSON modification then update | Menu option 4 + ID + Update        | Error message shown        |
| DT-09        | Termination during add operation     | Menu option 1 → name + Termination | JSON file data persistence |

Table 5.41

### • Summary

The test conditions and test cases cover the main functionality of the Contact Management System. Unit test cases focus on validating individual functions in the service layer, while integration test cases verify that the service and repository work together correctly. Dynamic and system test cases ensure that the program behaves correctly during actual execution and that the full user workflow is completed successfully.



## 6. Test Implementation and Logs

### 6.1 Test Execution Results

| Test ID | Test Case                    | Status | Actual Result                                                             |
|---------|------------------------------|--------|---------------------------------------------------------------------------|
| IT-01   | Add contact integration      | Pass   | Contact added through service was saved into the JSON file                |
| IT-02   | View contacts integration    | Pass   | Contacts were loaded from the JSON file and displayed correctly           |
| IT-03   | Search integration           | Pass   | Contact was loaded from file and found successfully through search        |
| IT-04   | Update integration           | Pass   | Updated contact data was saved and appeared correctly after reload        |
| IT-05   | Delete integration           | Pass   | Deleted contact was removed from the JSON file after reload               |
| UT-01   | Add valid contact            | Pass   | Contact was added successfully and stored with the correct name and phone |
| UT-02   | Add contact with empty name  | Pass   | Operation was rejected and no contact was added                           |
| UT-03   | Add contact with empty phone | Pass   | Operation was rejected and no contact was added                           |
| UT-04   | Generate unique ID           | Pass   | Multiple contacts were created with different IDs                         |
| UT-05   | Validate phone format        | Pass   | Invalid phone number was rejected                                         |
| UT-06   | Add duplicate contact        | Fail   | Current implementation accepted the duplicate contact                     |
| UT-07   | Search existing contact      | Pass   | Matching contact was returned                                             |
| UT-08   | Search partial name          | Pass   | Matching contacts were returned based on partial input                    |
| UT-09   | Search non-existing contact  | Pass   | No contacts were returned                                                 |
| UT-10   | Delete valid ID              | Pass   | Contact was removed successfully                                          |
| UT-11   | Delete invalid ID            | Pass   | Delete operation was rejected                                             |

| Test ID | Test Case            | Status | Actual Result                                |
|---------|----------------------|--------|----------------------------------------------|
| UT-12   | Update valid contact | Pass   | Contact information was updated successfully |
| UT-13   | Update invalid ID    | Pass   | Update operation was rejected                |

Table 6.11

- Dynamic Test Logs

| Test Case ID | Test Case                          | Execution Status | Actual Result                                                                                                                                                                         |
|--------------|------------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DT-01        | Add contact via menu               | Passed           | Contact was added successfully after entering a valid name and phone number                                                                                                           |
| DT-02        | View contacts                      | Passed           | All saved contacts were displayed correctly using menu option 2                                                                                                                       |
| DT-03        | Search contact                     | Passed           | Matching contact results were displayed after entering a valid search name                                                                                                            |
| DT-04        | Update contact                     | Passed           | Contact data was updated successfully using a valid ID                                                                                                                                |
| DT-05        | Delete contact                     | Passed           | Contact was removed successfully using a valid ID                                                                                                                                     |
| DT-06        | Invalid menu choice                | Passed           | System displayed an error message when input 99 was entered                                                                                                                           |
| DT-07        | Empty input handling               | Passed           | System displayed a validation error when empty name or phone input was entered                                                                                                        |
| DT-08        | Editing in JSON file, and updating | Failed           | System did not handle the manually edited JSON data, there were many contacts with the same ID. However, the system did not crash, but it updated the first contact with the ID given |
| DT-09        | Exiting while in operation         | Passed           | Previously saved JSON data remained persistent and was not corrupted after termination                                                                                                |

Table 6.12

- System Test Logs

| Test Case ID | Test Case               | Execution Status | Actual Result                                                                                                     |
|--------------|-------------------------|------------------|-------------------------------------------------------------------------------------------------------------------|
| ST-01        | Full workflow test      | Passed           | Add, view, search, update, and delete operations worked correctly in sequence                                     |
| ST-02        | Startup test            | Passed           | Program started successfully and loaded contacts from JSON, or created an empty contact list when no data existed |
| ST-03        | Data persistence test   | Passed           | Contact remained saved after restarting the program                                                               |
| ST-04        | End-to-end update test  | Passed           | Updated contact data was displayed correctly after viewing contacts                                               |
| ST-05        | End-to-end delete test  | Passed           | Deleted contact no longer appeared in the contact list                                                            |
| ST-06        | Menu navigation test    | Passed           | All menu options worked correctly and the program did not crash                                                   |
| ST-07        | JSON file recovery test | Passed           | System handled empty or missing JSON file without crashing, creating a new one if missing                         |

Table 6.13

- Test Cases Logs

|                               |                                                                                       |
|-------------------------------|---------------------------------------------------------------------------------------|
| TestingProj (18)              | 38 ms                                                                                 |
| <Empty Namespace> (18)        | 38 ms                                                                                 |
| ContactIntegrationTest (5)    | 32 ms                                                                                 |
| IT01_AddContactIntegration    | 7 ms GoogleTes...                                                                     |
| IT02_ViewContactsIntegration  | 6 ms GoogleTes...                                                                     |
| IT03_SearchIntegration        | 2 ms GoogleTes...                                                                     |
| IT04_UpdateIntegration        | 11 ms GoogleTes...                                                                    |
| IT05_DeleteIntegration        | 6 ms GoogleTes...                                                                     |
| ContactServiceUnitTest (13)   | 6 ms                                                                                  |
| UT01_AddValidContact          | < 1 ms GoogleTes...                                                                   |
| UT02_AddEmptyName             | < 1 ms GoogleTes...                                                                   |
| UT03_AddEmptyPhone            | < 1 ms GoogleTes...                                                                   |
| UT04_UniqueID                 | 1 ms GoogleTes...                                                                     |
| UT05_InvalidPhone             | < 1 ms GoogleTes...                                                                   |
| UT06_AddDuplicateContact      | 1 ms GoogleTes... #1 - Value of: duplicateAdd Actual: true Expected: false #2 - Va... |
| UT07_SearchExistingContact    | 1 ms GoogleTes...                                                                     |
| UT08_SearchPartialName        | 2 ms GoogleTes...                                                                     |
| UT09_SearchNonExistingContact | < 1 ms GoogleTes...                                                                   |
| UT10_DeleteValidID            | 1 ms GoogleTes...                                                                     |
| UT11_DeleteInvalidID          | < 1 ms GoogleTes...                                                                   |
| UT12_UpdateValidContact       | < 1 ms GoogleTes...                                                                   |
| UT13_UpdateInvalidID          | < 1 ms GoogleTes...                                                                   |

Fig 6.1

## 6.2 Defect Log

| Defect ID | Related Test Case | Defect Description                                                                         | Severity | Priority | Status |
|-----------|-------------------|--------------------------------------------------------------------------------------------|----------|----------|--------|
| DEF-01    | Static Review     | ContactConsole visibility                                                                  | Moderate | Moderate | Open   |
| DEF-02    | Static Review     | Void Initialize() present in code but not in class diagram                                 | Moderate | Moderate | Open   |
| DEF-03    | Static Review     | If load() fails or denied, no error is shown                                               | High     | High     | Open   |
| DEF-04    | Static Review     | System lacks duplicates checking before adding                                             | Moderate | Moderate | Open   |
| DEF-05    | Static Review     | If save() fails after add(), there is no rollback option                                   | High     | High     | Open   |
| DEF-06    | Static Review     | setName,Phone() does not show that it validates input, although it does in code            | Low      | Low      | Open   |
| DEF-07    | Static Review     | isValidPhone() does not validate length, therefore any number of characters can be entered | Low      | Low      | Open   |
| DEF-08    | UT-06             | System accepts duplicate contacts                                                          | Medium   | High     | Open   |
| DEF-09    | DT-08             | System does not handle manual data manipulation in JSON file, leading to corruption        | High     | High     | Open   |

Table 6.21

### • Final Defect Report Result

A total of **9 defects** were identified. The most important defects are the duplicate contact issue and the JSON manual editing issue. These should be fixed first because they affect data correctness and system reliability. Other issues, such as strict search validation and inefficient copying, should be improved to enhance usability and maintainability.

## 7. Conclusion

The testing process for the Contact Manager System was conducted using several testing techniques and levels to ensure software quality and requirement compliance. The performed tests helped identify defects related to validation, persistence, and functionality.

Overall, the system achieved acceptable results in most testing areas, while some issues were identified and documented for future improvements. The testing activities contributed to improving the reliability, usability, and maintainability of the application.

## 8. References

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

GoogleTest Team. (2025). *GoogleTest user's guide*. GitHub. <https://google.github.io/googletest/>

nlohmann, N. (2025). *JSON for Modern C++ documentation*. GitHub.

<https://json.nlohmann.me/>

Microsoft. (2025). *Visual Studio documentation*. Microsoft Learn.

<https://learn.microsoft.com/en-us/visualstudio/>

IEEE. (2013). *IEEE standard for software and system test documentation (IEEE Std 829-2008)*.

IEEE Standards Association. <https://standards.ieee.org/>

ISO/IEC/IEEE. (2017). *Systems and software engineering—Vocabulary (ISO/IEC/IEEE*

*24765:2017)*. ISO. <https://www.iso.org/>

The Open Source Initiative. (2025). *Introducing JSON*. <https://www.json.org/json-en.html>